

CESAR School - 2026.1

LEVEL UP RECIFE MEI

Grupo 7

OBJETIVO DO PROJETO

O Recife MEI é um aplicativo mobile focado no suporte a negócios que tem como objetivo democratizar e facilitar o processo de licitações e concorrências públicas para o MEI. A ferramenta serve como um apoio preparatório essencial para que o pequeno empreendedor aumente suas chances de vencer contratos com o governo com total segurança e conformidade.

O QUE FOI DESENVOLVIDO?

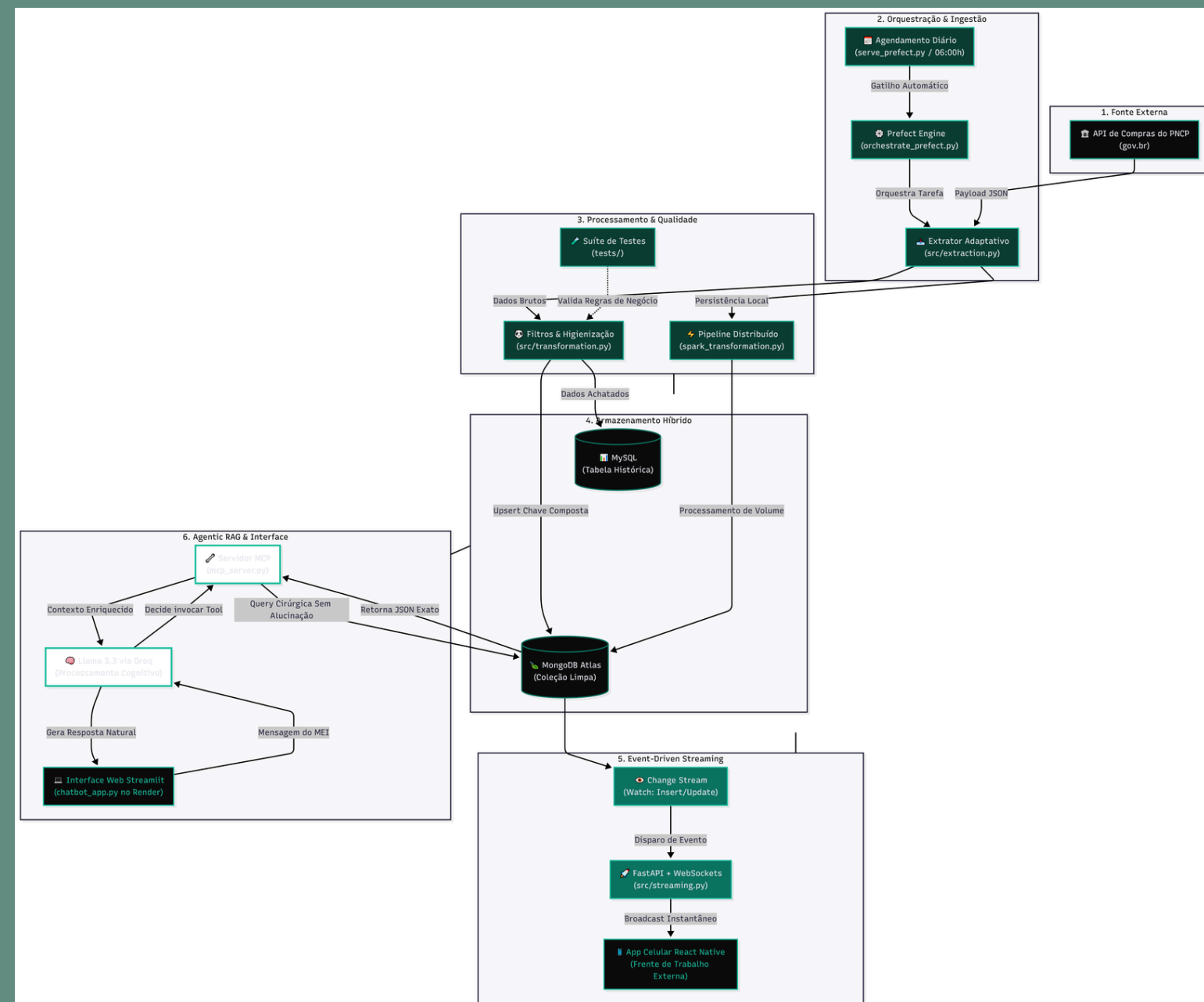
PROBLEMA

Milhões de MEIs ficam de fora das compras públicas por três motivos: as oportunidades estão espalhadas em portais complexos, os editais usam linguagem jurídica difícil e a gestão de prazos e documentos é manual e arriscada.

SOLUÇÃO

Nossa plataforma preenche essa lacuna: centraliza os editais do PNCP, traduz os requisitos para uma linguagem simples e organiza toda a jornada de participação. Construimos um produto completo e integrado de ponta a ponta, com pipeline de dados, API e aplicativo.

ARQUITETURA



A ORQUESTRAÇÃO E O CORAÇÃO DO ETL

PAPEL DO PREFECT

Responsável pela orquestração, automação e controle das etapas do ETL.

FLUXO PRINCIPAL

Coordena a execução do fluxo com janela móvel de 30 dias e rotina diária agendada.

EXTRAÇÃO E O DESAFIO DA TRANSFORMAÇÃO

DESAFIOS

- Extração da API PNCP dentro do contexto de paginação, timeouts, filtros de IBGE para Recife.
- Para transformação: o filtro de teto financeiro (R\$ 81.000 para MEI).
- Possível perda de dados aninhados.

SOLUÇÃO

- Criação de duas saídas pros dados:
 - Uma para dados completos/brutos (MongoDB).
 - Uma "achatada" apenas com colunas escalares (MySQL).

ARQUITETURA DE PERSISTÊNCIA HÍBRIDA

Paradigma Híbrido

Utilização de dois bancos de dados no projeto: Por quê?

MongoDB

- Coleta de dados direto da API PNCP
- A flexibilidade entre coleções brutas e limpas
- Banco de dados utilizado pelo streaming de dados

MySQL

- Tabela estruturada
- Focada em histórico
- Integrações analíticas.



DATA STREAMING & IA E PROTOCOLLO MCP

FLUXO ETL E CAMADAS DE DADOS

Da API pública até o chatbot: o pipeline organiza dados brutos, tratados e prontos para análise.



Bronze

Preserva o dado original do PNCP para rastreabilidade e reprocessamento.

Silver

Entrega dados higienizados, mantendo campos aninhados para a aplicação.

Gold

Concentra colunas escalares em MySQL para consulta relacional e análise.

ETL ORQUESTRADO COM PREFECT

O Prefect dá ordem, logs, retries e agendamento ao pipeline, evitando execução manual solta.

1. Janela móvel

A data inicial e final são calculadas em runtime. O fluxo usa os últimos N dias e termina em amanhã, sem datas fixas no código.

2. Tasks do fluxo

Extração, carga bruta, transformação, carga limpa e carga gold aparecem como etapas explícitas dentro do flow.

3. Operação diária

O projeto pode rodar com agendamento cron diário às 06:00 via Prefect, mantendo a rotina automatizada.

Código principal

```
orchestrate_prefect.py:17 def janela_dados(...)  
orchestrate_prefect.py:36 fim = hoje + timedelta(days=1)  
orchestrate_prefect.py:175 def etl_pncp_flow(...)  
orchestrate_prefect.py:190 run_id = str(uuid.uuid4())
```

Sequência das etapas

201 task_extrair(...)	-> PNCP
211 task_carregar_mongodb(...)	-> Bronze
213 task_transformar(...)	-> Silver
226 task_carregar_mysql(...)	-> Gold

PERSISTÊNCIA HÍBRIDA

MongoDB e MySQL têm papéis diferentes: flexibilidade documental e estrutura relacional.

MongoDB | Bronze

Recebe documentos brutos completos, próximos do retorno da API. Se a collection for apagada, o upsert recria os registros quando o ETL rodar novamente.

Bronze

```
src/database.py:64
def carregar_mongodb(...)
src/database.py:82
upsert=True
```

MongoDB | Silver

Recebe os dados limpos sem perder estruturas aninhadas como rgfo, unidade e links oficiais, que são úteis para a aplicação.

Silver

```
src/database.py:93
def carregar_mongodb_limpos(...)
src/database.py:123
upsert=True
```

MySQL | Gold

Recebe apenas colunas escalares, porque o modelo relacional é melhor para tabelas, consultas e integrações analíticas.

Gold

```
src/database.py:59
def carregar_mysql(...)
src/database.py:62
df.to_sql(...)
```

TRANSFORMAÇÃO DOS DADOS E SPARK

A transformação limpa os registros para uso imediato e o PySpark aparece como caminho escalável.

Transformação padrão

O TransformadorDados padroniza campos, aplica regras como filtro de valor, deriva município e preserva os campos importantes para a camada Silver.

Transformador do projeto

```
src/transformation.py:30 COLUNAS_ESCALARES
src/transformation.py:80 valorTotalEstimado
src/transformation.py:93 municipioNome
src/transformation.py:125 return self._higienizar(...)
```

Transformação com PySpark

O Spark I? do MongoDB, transforma o DataFrame e grava novamente no MongoDB e no MySQL. ? uma alternativa mais adequada quando o volume cresce.

Spark

```
spark_transformation.py:65 SparkSession.builder
spark_transformation.py:90 spark.read.format('mongodb').load()
spark_transformation.py:158 _upsert_key
spark_transformation.py:223 colunas_relacionais
```

MCP: CAMADA DE FERRAMENTAS

O servidor MCP transforma regras de consulta em ferramentas seguras para o chatbot.



Ferramentas expostas

Consultar MongoDB, resumir contratações, buscar por número, listar municípios, checar status do pipeline e filtrar por valor ou objeto.

Referências do MCP

```
mcp_server.py:12  mcp = FastMCP(...)
mcp_server.py:24  MONGO_COLLECTION = 'contratacoes_brutas'
mcp_server.py:33  def consultar_mongodb(...)
mcp_server.py:72  def resumir_contratacoes_mongodb()
mcp_server.py:257 mcp.run(transport='stdio')
```

CHATBOT INTEGRADO AO PROJETO

A interface Streamlit conecta usuário, modelo Llama/Groq e ferramentas MCP para responder com dados do projeto.

Interface

O Streamlit controla tela, histórico, campo de pergunta e apresentação das respostas para o usuário.

Modelo

O Groq/Llama interpreta a pergunta e decide automaticamente quando precisa chamar uma ferramenta.

Ferramentas

O MCP executa a consulta no MongoDB e devolve o resultado para o modelo montar a resposta final.

Fluxo no Streamlit

```
chatbot_app.py:549 user_input = st.chat_input(...)
chatbot_app.py:581 tools = asyncio.run(fetch_mcp_tools())
chatbot_app.py:583 chat_with_groq(active_input, history, tools)
```

Function calling

```
chatbot_app.py:290 async def fetch_mcp_tools()
chatbot_app.py:317 async def call_mcp_tool(...)
chatbot_app.py:343 client = Groq(api_key=api_key)
chatbot_app.py:361 kwargs['tool_choice'] = 'auto'
```

CESAR School - 2026.1

Obrigado!

Grupo 7